UNIVERSITY OF THE PUNJAB

DEPARTMENT OF DATA SCIENCE

Quaid-i-Azam Campus, Lahore, Pakistan

COURSE PROJECT FOR SE

FULL PROJECT DELIVERABLE D3

System Requirements Specification (Ludo Game Component)

Course Title:	Software Engineering & Project Management (Fall 2026)
Group Name:	_____
Document Version:	v1 (Final Release)
Instructor:	Dr. Shahid Manzoor
Section:	BSDSF24M
Team Members:	• **Ahad Ali Saeed** (BSDSF24M031) — *bsdsf24m031@pucit.edu.pk* • **Sufyan Ahmad** (BSDSF24M018) — *bsdsf24m018@pucit.edu.pk* • **Abdullah Ayub** (BSDSF24M029) — *bsdsf24m029@pucit.edu.pk*

Date of Submission: November 4, 2026

1. Scope Statement

This document specifies the system requirements of the digital Ludo game our team has built for the Software Engineering and Project Management course (Fall 2026) at the Department of Data Science, University of the Punjab. It turns the 18 user stories of Deliverable D2 (REQ-PLAYER1 to REQ-PLAYER18) into precise functional and non-functional system requirements that other teams can use to understand, integrate and test our component. It builds on the group agreement in D1 and the elicitation in D2.

The component is a web-based Ludo board-race game for 2 to 4 players. Each seat can be a human or a computer player, so Human vs Human and Human vs Computer games are both possible. The game offers two modes: Single Player (one human against computer players) and Multiplayer (the user chooses how many humans play, and the remaining seats are computer players). It follows the standard Ludo rules, with a few explicit rules for extra turns and consecutive sixes (Section 2.3). The game enforces valid moves, rolls a fair virtual dice, tracks turns and each player's progress, handles captures and announces the winner automatically.

The game is built with HTML, CSS and JavaScript and runs completely in the browser, so it needs no installation, no plug-in and no server; its front end (the UI Layer) and its back end (the game logic modules) are both described in Section 3.4. Its main differentiators are: (1) a fast computer opponent that plays automatically within one second using a priority-based move choice; (2) clear messages for invalid moves that never disturb the board; (3) a live progress panel with the number of pieces at start, on the board and finished for every player; and (4) Restart, New Game and Quit available at any point during play.

1.1 Client and elicitation context (from D2)

The teaching assistant team acts as the primary product client. During Sprint 1 elicitation the client asked for a web-based board-race game for 2 to 4 players inspired by traditional Ludo [1], with full support for Human vs Human and Human vs Computer modes, automatic enforcement of the movement rules, automatic turn management, real-time progress indicators and reliable winner identification.

1.2 User classes and personas

Requirements were designed around the two user classes defined in D2:

- **Hassan (19, casual player)** — a university student who plays casual matches during breaks. He rarely reads rule manuals and needs intuitive controls, simple seat selection, automatic execution of single moves and a visual indicator of whose turn it is at all times.
- **Mariam (22, experienced player)** — a competitive board-game enthusiast who values fairness and strict rule compliance. She expects unbiased random dice, correct capture mechanics, transparent progress tracking and an unambiguous winner announcement.

1.3 Group contact information

Team member	Student ID	Contact e-mail
Ahad Ali Saeed	BSDSF24M031	bsdsf24m031@pucit.edu.pk
Sufyan Ahmad	BSDSF24M018	bsdsf24m018@pucit.edu.pk
Abdullah Ayub	BSDSF24M029	bsdsf24m029@pucit.edu.pk

Instructor: Dr. Shahid Manzoor • Section: BSDSF24M • Department of Data Science, University of the Punjab, Quaid-i-Azam Campus, Lahore, Pakistan.

2. Definitions

This section defines the terms used in the rest of the document, the game modes, the rules of the game and the board specification. The rules are the standard Ludo rules [1] plus the choices our team made. The client has still to confirm the detailed rules (see D2), so rules marked as preliminary may change.

2.1 Terms

Term	Definition
Ludo	A race board game for 2 to 4 players in which each player moves four pieces from the start area around the board to the final destination before the others.
Board	The cross-shaped playing area made of the main track, four coloured home columns, four start areas and a centre.
Track	The shared circular path of 52 squares that every piece follows.
Piece	A token of one player's colour. Each player has 4 pieces.
Start area	The area of a player's colour where pieces wait until a six is rolled. A piece in it is called "at start".
Start square	The square of the track where a piece enters after leaving the start area. Each colour has its own.
Home column	The last 6 squares of a player's path, reachable only by that player's pieces.
Final destination	The centre square at the end of the home column. A piece that reaches it is "finished".
Safe square	A track square where a piece cannot be captured: the four start squares and the four star squares.
Dice	A virtual six-sided dice that gives a random integer from 1 to 6.
Roll	One use of the dice by the player whose turn it is.
Turn	The period in which one player rolls and moves. A turn can include extra rolls.
Extra turn	An additional roll given to the same player after a six, a capture or a piece finishing.
Consecutive sixes	Sixes rolled one after another by the same player. Every complete group of three consecutive sixes is cancelled (rule 14).
Capture	Landing on a square occupied by an opponent's piece (not a safe square), which sends that piece back to its start area.
Valid move	A move that follows all rules in Section 2.3.
Invalid move	A move that breaks a rule. The game rejects it and shows a message.
Player	A participant of the game, either human or computer.
Human player	A person who plays using mouse or touch input and has a name entered at setup.
Computer player (CPU)	A player controlled by the game that rolls and chooses its moves automatically.
Seat	One of the 2 to 4 player positions in a game. Each seat has a colour and is set as human or computer.
Active player	The player whose turn it is. Only the active player can roll and move.
Progress	For each player, the number of pieces at start, on the board and finished.
Winner	The first player whose 4 pieces are all finished.
Restart	Starting the current game again with the same settings.
New game	Starting a fresh game from setup, with new settings.
Game State	The data that describes the game at any moment: seats, piece positions, current turn, count of consecutive sixes, progress and winner.
PRNG	Pseudo-random number generator: the software routine that produces the dice values.

2.2 Game modes

Mode	Description
Single Player	One human plays against computer players. The user chooses the number of players (2, 3 or 4); the human takes one seat and all other seats are computer players.
Multiplayer	The user is asked how many human players take part (from 2 up to the chosen number of players). The user also chooses the total number of players (2 to 4) and any remaining seats are computer players. Each human enters a name.

In both modes each seat is either human or computer, as required by REQ-PLAYER2 and REQ-PLAYER3 of D2. With 2 players the seats sit in opposite corners of the board.

2.3 Rules of the game

Setup

1. Each player has 4 pieces, all in the start area at the beginning.
2. The seats play in a fixed clockwise order. The first player is chosen at random.

Turn and movement

3. On their turn the player rolls the dice once.
4. A piece at start can leave only when a six is rolled. It is placed on its own start square.
5. A piece on the track moves clockwise by the number rolled.
6. If only one move is possible, the game makes it automatically. If several pieces can move, a human player chooses one and a computer player chooses by itself.
7. If no move is possible, the turn passes to the next player.
8. A piece must reach the final destination with an exact roll. If the roll is too high for a piece, that piece cannot move.

Capture and safe squares

9. Landing on a square with an opponent's piece captures it, and the captured piece returns to its start area.
10. Pieces on safe squares cannot be captured. Pieces can share a safe square.
11. A player never captures their own pieces. Two pieces of the same player may share a square, and there are no blocking pairs: a pair of pieces does not stop opponents from passing or landing.
12. Pieces in the home column cannot be captured.

Extra turns and consecutive sixes

13. A player rolls again after rolling a six, after a capture and after a piece reaches the final destination. Extra rolls earned by a capture or a finished piece are taken after the rolls of the current sequence have been played.
14. Sixes are counted while they keep coming. When a different number appears, every complete group of three consecutive sixes is cancelled and only the remaining rolls are played. Examples: 6, 6, 6, 4 plays only the 4. Six sixes followed by 5 plays only the 5. Nine sixes followed by 2 plays only the 2. One or two leftover sixes are played normally, for example 6, 6, 6, 6, 3 plays one 6 and then the 3.

End of the game

15. The first player to bring all 4 pieces to the final destination wins. The game stops and the winner is announced.
16. The players can restart the game, start a new one or quit at any point during play and after the game has ended.

2.4 Board and path specification

The values below are the default parameters of the game. They are stored as named settings in one place (NFR 5.6.1), so they can be changed without touching the rest of the code.

Parameter	Default value
Track squares	52 squares, numbered 0 to 51 clockwise.
Start squares	One per colour, 13 squares apart (squares 0, 13, 26 and 39 for the four colours).
Star squares	The square 8 positions clockwise after each start square (squares 8, 21, 34 and 47).
Safe squares	8 in total: the 4 start squares and the 4 star squares.
Path of one piece	Start square (step 0), 50 further track squares (steps 1 to 50), then the 6 squares of the home column (steps 51 to 56). Step 56 is the final destination.

Parameter	Default value
Exact finish	A piece at step p can use a roll r only if p + r ≤ 56.
Players / seats	2 to 4 players; 4 pieces each; colours red, green, yellow, blue; names of 1 to 12 characters.

3. Components

The Ludo game is one web application that runs completely in the browser. It is split into six logical components. The Game Controller is the hub: it receives the player's actions from the UI Layer, asks the other components for results and keeps the Game State up to date. A human player only talks to the UI Layer. The Game Controller asks the Dice Module for a roll, the Rules Engine for the valid moves and their result, and the CPU Player for the move of a computer seat. Arrows with two heads mean that information flows both ways.

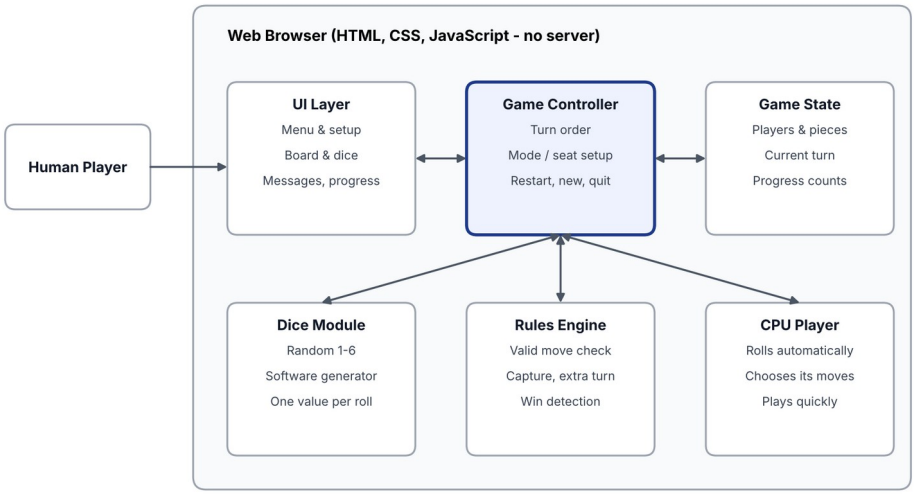


Figure 1: Simplified component diagram of the Ludo game

3.1 Architecture rationale

The architecture is a client-side, controller-centred design: the game has no server, no database and no network calls. Rules, state and drawing are kept in separate components, so each can be tested alone (NFR 5.6.1). The CPU Player uses exactly the same Dice Module and Rules Engine as a human seat, which guarantees fair play (NFR 5.7.1).

3.2 Component descriptions and interfaces

Component	Description	Input	Output
UI Layer	Shows the main menu, the setup screens, the rules summary, the board, the pieces, the dice, whose turn it is, the progress of each player and all messages (invalid move, winner). It redraws after every action.	Player clicks or taps; Game State	Player actions for the Game Controller; the updated screen
Game Controller	Runs the game. It handles mode and seat setup, turn order, extra turns, restart, new game and quit, and coordinates all other components.	Player actions; results from the Dice Module, Rules Engine and CPU Player	Updates to the Game State; requests to other components; messages for the UI Layer
Game State	Stores the seats (name, colour, human or computer), the position of every piece, the current turn, the count of consecutive sixes, the progress of each player and the winner.	Updates from the Game Controller	Current state for the Game Controller and the UI Layer
Dice Module	Gives a random integer from 1 to 6 for every roll, using a software random number generator.	Roll request	Dice value

Component	Description	Input	Output
Rules Engine	Applies the rules of Section 2.3. It lists the valid moves for a roll, checks a chosen move, detects captures, extra turns, the consecutive-sixes rule and the win condition.	Game State; dice value; chosen piece	Valid moves; result of a move (new positions, capture, extra turn, winner); reason for an invalid move
CPU Player	Plays the turn of a computer seat automatically and quickly by choosing one of the valid moves with the priority: finish > capture > leave start > advance the furthest piece.	Valid moves; Game State	Chosen piece

3.3 Data flow of a player action

Figure 2 shows how the components cooperate when a human player rolls the dice and moves a piece. The same sequence is used for a computer seat, except that the CPU Player replaces the human clicks in steps 1, 8 and 9.

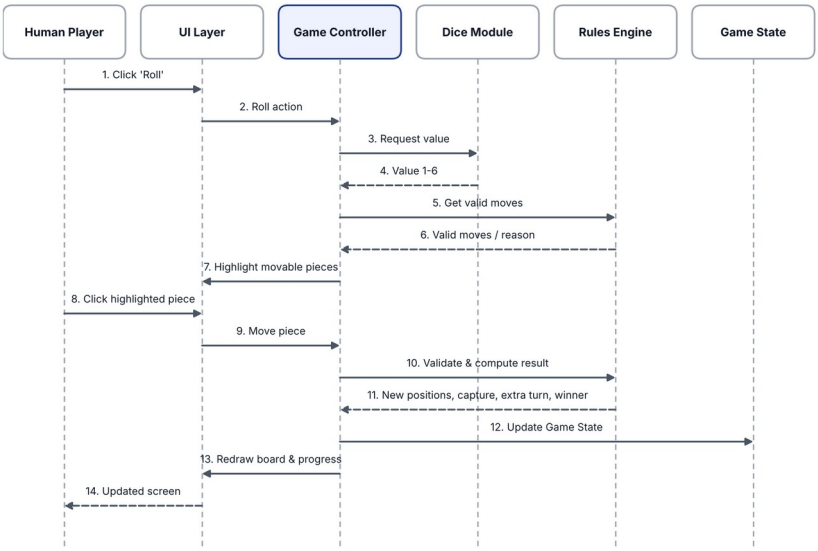


Figure 2: Subsystem interaction and data flow for a roll and a move

3.4 Front end and back end

Although the game has no server, it is clearly divided into a front end and a back end, both delivered in the same web page. The **front end** is everything the player sees and touches. The **back end** is the game logic that decides what is allowed and what happens next. The two sides only exchange player actions and state updates through the Game Controller, so the front end never applies a rule and the back end never draws anything. The front end is the UI Layer written in HTML, CSS and JavaScript. The back end is the Game Controller, Rules Engine, Dice Module, CPU Player and Game State, written as plain JavaScript modules that keep all data in browser memory. There is no server side: no server, database, network request or stored account (NFR 5.2.1); the page is only hosted as static files so that it can be opened from a link.

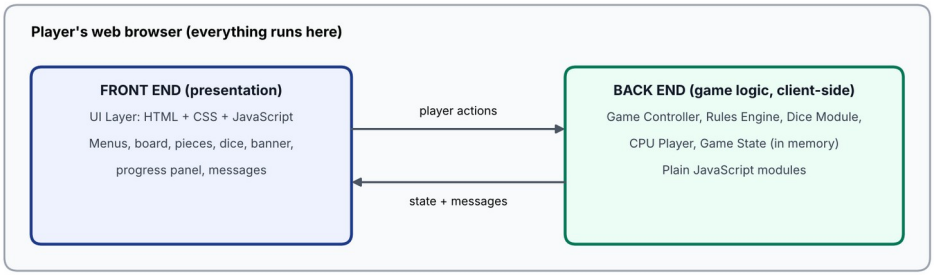


Figure 3: Front end and back end inside the browser

3.5 Flow of one turn

Figure 4 summarises the life of a turn, including the consecutive-sixes rule, the automatic single move, extra rolls and the end of the game.

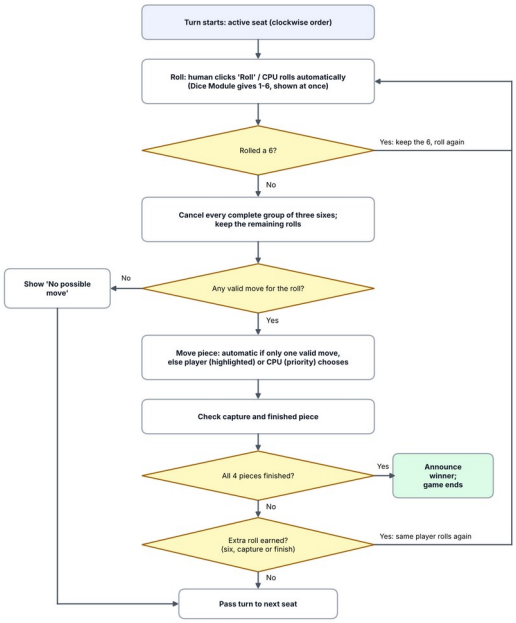


Figure 4: Flow of one turn

3.6 Interaction with other systems

Our team does not yet know the other components of the course project, but we expect the Ludo game to be used as a game component. It runs from a single web page, so another system only needs to open that page. If a platform needs the result, the game can provide the final result (winner name or colour, and the number of players) when a game ends. This is an expectation and not a requirement yet.

4. Functional Requirements

The functional requirements below describe what the Ludo game must do. Each one comes from one or more user stories of D2 (REQ-PLAYER1 to REQ-PLAYER18) and uses the template of the course. The rules they refer to are in Section 2.3. A traceability table in Section 6 shows which requirement covers which user story. Requirement IDs have the form REQ-PLAYERn.m (n is the D2 user story it refines), preconditions are PRE-x.y-n and postconditions POST-x.y-n; non-functional IDs have the form NFR s.n.

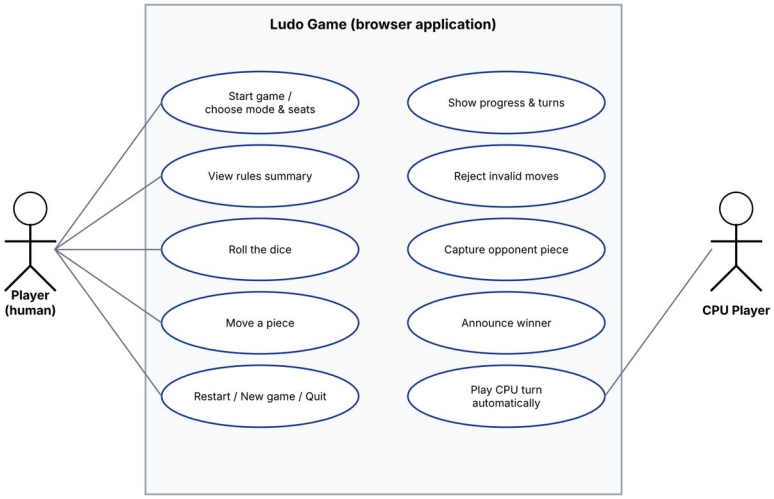


Figure 5: Use case diagram of the Ludo game

4.1 REQ-PLAYER1.1 Start a new game

Item	Content
ID and Name	REQ-PLAYER1.1 Start a new game
Created by	Ahad Ali Saeed
Date created	2026-11-04
User requirement	REQ-PLAYER1 (Start Game)
Primary actor	Player
Secondary actors	Game Controller, UI Layer
Description	The system shall show a main menu with the options "New Game", "Rules" and "Quit". Choosing "New Game" shall open the mode selection screen (REQ-PLAYER2.1).
Trigger	The player opens the game page or clicks "New Game".
Preconditions	PRE-1.1-1 The game page is loaded in the browser. PRE-1.1-2 No game is in progress.
Postconditions	POST-1.1-1 The mode selection screen is shown. POST-1.1-2 Settings of any earlier game are cleared.
Normal flow	1. The player opens the game page. 2. The system shows the main menu. 3. The player clicks "New Game". 4. The system shows the mode selection screen.
Alternative flows	A1. The player clicks "Rules" and the flow continues in REQ-PLAYER5.1. A2. The player clicks "Quit" and the flow continues in REQ-PLAYER16.1.
Exceptions	E1. JavaScript is disabled in the browser. The page shows the message "JavaScript is required to play this game."
Diagrams	Figure 5 (use case diagram).

4.2 REQ-PLAYER2.1 Choose game mode and seats

Item	Content
ID and Name	REQ-PLAYER2.1 Choose game mode and seats
Created by	Ahad Ali Saeed
Date created	2026-11-04
User requirement	REQ-PLAYER2, REQ-PLAYER3, REQ-PLAYER4 (Mode, Seats, Names)
Primary actor	Player
Secondary actors	Game Controller, Game State
Description	The system shall offer two modes, Single Player and Multiplayer. In both modes the number of players is 2, 3 or 4 and every seat is either human or computer. In Single Player one seat is human and all other seats are computer players. In Multiplayer the system asks how many players are human (from 2 up to the number of players) and the remaining seats are computer players. The system shall ask for a name of 1 to 12 characters for every human seat and shall give each seat a colour (red, green, yellow, blue). With 2 players the seats sit in opposite corners.
Trigger	The player has clicked "New Game".
Preconditions	PRE-2.1-1 The mode selection screen is shown.
Postconditions	POST-2.1-1 Mode, number of players, seat types, names and colours are stored in the Game State. POST-2.1-2 The board is shown with 4 pieces of every player in the start area. POST-2.1-3 The first player is chosen at random.
Normal flow	1. The player chooses Single Player or Multiplayer. 2. The player chooses the number of players (2, 3 or 4). 3. In Multiplayer the player chooses the number of human players.

Item	Content
	4. The player enters a name for every human seat. 5. The player clicks "Start Game". 6. The system creates the game and shows the board.
Alternative flows	A1. In Single Player step 3 is skipped and the number of human players is 1. A2. The player clicks "Back" and returns to the main menu.
Exceptions	E1. Two human names are the same. The system shows "Each player needs a different name." and asks again. E2. A name is empty. The system uses "Player 1", "Player 2" and so on. E3. A name is longer than 12 characters. The system shows "A name can have at most 12 characters."
Diagrams	Figure 5 (use case diagram).

4.3 REQ-PLAYER5.1 View the rules summary

Item	Content
ID and Name	REQ-PLAYER5.1 View the rules summary
Created by	Ahad Ali Saeed
Date created	2026-11-04
User requirement	REQ-PLAYER5 (Read Rules)
Primary actor	Player
Secondary actors	UI Layer
Description	The system shall show a short summary of the rules of Section 2.3 from the main menu and from the game screen. Opening the summary during a game shall not change the game.
Trigger	The player clicks "Rules".
Preconditions	PRE-5.1-1 The main menu or the game screen is shown.
Postconditions	POST-5.1-1 The previous screen is shown again after closing the summary. POST-5.1-2 The Game State is unchanged.
Normal flow	1. The player clicks "Rules". 2. The system shows the rules summary. 3. The player clicks "Close". 4. The system returns to the previous screen.
Alternative flows	None.
Exceptions	None.
Diagrams	None.

4.4 REQ-PLAYER6.1 Roll the dice

Item	Content
ID and Name	REQ-PLAYER6.1 Roll the dice
Created by	Sufyan Ahmad
Date created	2026-11-04
User requirement	REQ-PLAYER6 (Roll Dice), REQ-PLAYER11 (Six Activation & Re-roll)
Primary actor	Player (human or CPU)
Secondary actors	Dice Module, Rules Engine, Game Controller
Description	The active human player shall roll the dice with a "Roll" button and a computer seat shall roll automatically. Every roll is a random integer from 1 to 6 given by the Dice Module, each value equally likely, and is shown at once. After a six, a capture or a finished piece the same player rolls again. Rolls

Item	Content
	of six are collected while sixes keep coming. When a different number appears, every complete group of three sixes is cancelled and only the remaining rolls are played (rule 14 of Section 2.3).
Trigger	The active human player clicks "Roll", or the turn of a computer seat starts.
Preconditions	PRE-6.1-1 A game is in progress. PRE-6.1-2 It is the turn of this player. PRE-6.1-3 The player has not yet rolled in this step.
Postconditions	POST-6.1-1 The dice value is shown. POST-6.1-2 The count of consecutive sixes in the Game State is updated. POST-6.1-3 The "Roll" button is disabled until the roll has been played.
Normal flow	1. The player clicks "Roll". 2. The system asks the Dice Module for a value. 3. The system shows the value. 4. If the value is 6, the system stores it and lets the player roll again (back to step 1). 5. Otherwise the system cancels every complete group of three sixes in the stored rolls and keeps the rest, including this value. 6. The system plays the kept rolls one after the other (REQ-PLAYER7.1).
Alternative flows	A1. A computer seat performs steps 1 and 2 automatically. A2. Examples: 6, 6, 6, 4 keeps only the 4. Six sixes then 5 keeps only the 5. 6, 6, 6, 6, 3 keeps one 6 and the 3.
Exceptions	E1. The dice value cannot be generated. The system shows "Roll failed, please roll again." and keeps the button enabled.
Diagrams	Figure 2 (interaction diagram) and Figure 4 (turn flow).

4.5 REQ-PLAYER7.1 Move a piece

Item	Content
ID and Name	REQ-PLAYER7.1 Move a piece
Created by	Sufyan Ahmad
Date created	2026-11-04
User requirement	REQ-PLAYER7 (Move Piece), REQ-PLAYER11 (Six Activation & Re-roll)
Primary actor	Player
Secondary actors	Rules Engine, Game Controller, Game State
Description	After a roll the system shall find all valid moves. If only one move is possible it shall be made automatically. If there are several, the system shall highlight the pieces that can move and the player chooses one, so no counting of squares is needed. A six lets a piece leave the start area and be placed on its start square. A piece moves clockwise by the dice value and needs an exact value to reach the final destination.
Trigger	A dice value is waiting to be played.
Preconditions	PRE-7.1-1 A game is in progress. PRE-7.1-2 A dice value is waiting to be played by the active player.
Postconditions	POST-7.1-1 The piece position is updated in the Game State. POST-7.1-2 The board is redrawn. POST-7.1-3 The turn continues (extra roll) or passes to the next player.
Normal flow	1. The system asks the Rules Engine for the valid moves. 2. If there is one valid move, the system makes it. 3. If there are several, the system highlights the pieces that can move. 4. The player clicks one highlighted piece. 5. The system moves the piece square by square. 6. The system checks for a capture (REQ-PLAYER10.1) and for a finished piece, then handles the

Item	Content
	turn (REQ-PLAYER8.1).
Alternative flows	A1. There is no valid move. The system shows "No possible move." and the turn passes to the next player. A2. A computer seat chooses its piece itself (REQ-PLAYER13.1).
Exceptions	E1. The player clicks a piece that cannot move. The flow continues in REQ-PLAYER9.1.
Diagrams	Figure 2 (interaction diagram) and Figure 4 (turn flow).

4.6 REQ-PLAYER8.1 Show and manage turns

Item	Content
ID and Name	REQ-PLAYER8.1 Show and manage turns
Created by	Abdullah Ayub
Date created	2026-11-04
User requirement	REQ-PLAYER8 (Turn Indicator)
Primary actor	Player
Secondary actors	Game Controller, UI Layer
Description	The system shall keep a clockwise turn order, show the name and colour of the active player on screen at all times and allow only the active player to roll and move. When a turn ends without an extra roll, the next seat becomes active.
Trigger	The game starts or a turn ends.
Preconditions	PRE-8.1-1 A game is in progress.
Postconditions	POST-8.1-1 Exactly one player is marked as active. POST-8.1-2 The controls of all other players are disabled.
Normal flow	1. A turn ends. 2. The Game Controller selects the next seat clockwise. 3. The Game State stores the new current turn. 4. The UI Layer highlights the name and shows "[Name]'s turn". 5. If the new seat is a computer player, REQ-PLAYER13.1 starts.
Alternative flows	A1. The player gets an extra roll. The same player stays active and the system shows "[Name] rolls again".
Exceptions	E1. A player who is not active clicks "Roll" or a piece. The click is ignored and the system shows "It is [Name]'s turn."
Diagrams	Figure 4 (turn flow).

4.7 REQ-PLAYER9.1 Reject invalid moves

Item	Content
ID and Name	REQ-PLAYER9.1 Reject invalid moves
Created by	Sufyan Ahmad
Date created	2026-11-04
User requirement	REQ-PLAYER9 (Invalid Move Feedback)
Primary actor	Player
Secondary actors	Rules Engine, UI Layer
Description	The system shall allow only valid moves (Section 2.3). When a player tries an invalid action, the system shall leave the game unchanged and show a clear message that says why the action is not allowed, so the player can correct it.
Trigger	A human player clicks a piece that cannot move with the current dice value, or clicks "Roll" when a roll

Item	Content
	is not allowed.
Preconditions	PRE-9.1-1 A game is in progress. PRE-9.1-2 It is the turn of the player.
Postconditions	POST-9.1-1 The board and the Game State are unchanged. POST-9.1-2 A message is shown. POST-9.1-3 The player can choose again.
Normal flow	1. The player clicks a piece. 2. The Rules Engine checks the move. 3. The move is not valid. 4. The UI Layer shows the reason, for example "You need a six to leave the start.", "You need the exact number to finish." or "This is not your piece." 5. The system waits for a new choice.
Alternative flows	A1. The player clicks "Roll" a second time before playing the roll. The click is ignored and the system shows "You have already rolled."
Exceptions	None.
Diagrams	Figure 4 (turn flow).

4.8 REQ-PLAYER10.1 Capture an opponent piece

Item	Content
ID and Name	REQ-PLAYER10.1 Capture an opponent piece
Created by	Abdullah Ayub
Date created	2026-11-04
User requirement	REQ-PLAYER10 (Capture Piece)
Primary actor	Player
Secondary actors	Rules Engine, Game State
Description	When a piece ends its move on a track square that is not safe and holds opponent pieces, the system shall return every opponent piece on that square to its start area and give the capturing player an extra roll. Pieces on safe squares and in home columns cannot be captured.
Trigger	A piece finishes a move on a track square.
Preconditions	PRE-10.1-1 A game is in progress. PRE-10.1-2 A move has just been made.
Postconditions	POST-10.1-1 Captured pieces are in the start area of their owners. POST-10.1-2 The progress of the owners is updated. POST-10.1-3 The capturing player has an extra roll.
Normal flow	1. The move ends. 2. The Rules Engine checks the square. 3. An opponent piece is on it and the square is not safe. 4. The system returns that piece to its start area. 5. The system shows "[Name] captured a piece of [Name]." 6. The capturing player rolls again.
Alternative flows	A1. The square is safe. No capture happens and the pieces share the square. A2. Only the player's own pieces are on the square. Nothing happens.
Exceptions	None.
Diagrams	Figure 4 (turn flow).

4.9 REQ-PLAYER12.1 Show progress and update the screen

Item	Content
ID and Name	REQ-PLAYER12.1 Show progress and update the screen
Created by	Abdullah Ayub
Date created	2026-11-04
User requirement	REQ-PLAYER12 (Progress), REQ-PLAYER14 (UI Speed)
Primary actor	Player
Secondary actors	Game Controller, Game State, UI Layer
Description	The system shall show for every player the number of pieces at start, on the board and finished. After every action (roll, move, capture, restart) the system shall update the board, the pieces, the dice and the progress panel at once.
Trigger	Any action changes the Game State.
Preconditions	PRE-12.1-1 A game is in progress.
Postconditions	POST-12.1-1 The screen shows the current Game State. POST-12.1-2 For every player, pieces at start + on the board + finished = 4.
Normal flow	1. An action changes the state. 2. The Game Controller updates the Game State. 3. The UI Layer redraws the board, the pieces and the dice. 4. The UI Layer updates the progress panel of every player.
Alternative flows	None.
Exceptions	None.
Diagrams	Figure 2 (interaction diagram), steps 12 to 14.

4.10 REQ-PLAYER13.1 Computer player turn

Item	Content
ID and Name	REQ-PLAYER13.1 Computer player turn
Created by	Sufyan Ahmad
Date created	2026-11-04
User requirement	REQ-PLAYER13 (CPU Speed)
Primary actor	Computer player
Secondary actors	Dice Module, Rules Engine, Game Controller
Description	When it is the turn of a computer seat, the system shall roll, choose one of the valid moves and make it without any input from a human, and shall repeat this for extra rolls. The computer chooses by a simple priority: finish a piece, capture an opponent piece, bring a piece out of the start area, otherwise move the piece that is furthest ahead. Human input is disabled during the turn.
Trigger	The next active seat is a computer player.
Preconditions	PRE-13.1-1 A game is in progress. PRE-13.1-2 The active seat is a computer player.
Postconditions	POST-13.1-1 The turn of the computer has been played completely. POST-13.1-2 The turn passes to the next seat.
Normal flow	1. The Game Controller starts the turn. 2. The Dice Module gives a value. 3. The CPU Player asks the Rules Engine for the valid moves. 4. The CPU Player chooses a move by the priority above. 5. The system makes the move and shows it. 6. For an extra roll the steps repeat. 7. The turn passes to the next seat.

Item	Content
Alternative flows	A1. There is no valid move. The system shows "[Name] has no possible move." and the turn passes.
Exceptions	None.
Diagrams	Figure 4 (turn flow).

4.11 REQ-PLAYER15.1 Announce the winner

Item	Content
ID and Name	REQ-PLAYER15.1 Announce the winner
Created by	Ahad Ali Saeed
Date created	2026-11-04
User requirement	REQ-PLAYER15 (Announce Winner)
Primary actor	Player
Secondary actors	Rules Engine, Game Controller, UI Layer
Description	After every move the system shall check whether all 4 pieces of the player are at the final destination. If so, it shall stop the game, show the name and colour of the winner and offer "Restart", "New Game" and "Quit". No further move is possible after that.
Trigger	A piece reaches the final destination.
Preconditions	PRE-15.1-1 A game is in progress. PRE-15.1-2 A piece has just reached the final destination.
Postconditions	POST-15.1-1 The game is marked as finished. POST-15.1-2 The winner is shown. POST-15.1-3 The "Roll" button is disabled.
Normal flow	1. A piece reaches the final destination. 2. The Rules Engine checks the 4 pieces of the player. 3. All 4 are finished. 4. The Game Controller marks the game as finished. 5. The UI Layer shows "[Name] wins!" with the end options.
Alternative flows	A1. Not all pieces are finished. The game continues and the player rolls again (rule 13 of Section 2.3).
Exceptions	None.
Diagrams	Figure 4 (turn flow).

4.12 REQ-PLAYER16.1 Restart, start a new game and quit

Item	Content
ID and Name	REQ-PLAYER16.1 Restart, start a new game and quit
Created by	Abdullah Ayub
Date created	2026-11-04
User requirement	REQ-PLAYER16, REQ-PLAYER17, REQ-PLAYER18 (Restart, New Game, Quit)
Primary actor	Player
Secondary actors	Game Controller, Game State
Description	The game screen shall always show the buttons "Restart", "New Game" and "Quit". "Restart" starts the current game again with the same settings, all pieces at start and a new random first player. "New Game" opens the mode selection screen (REQ-PLAYER2.1). "Quit" ends the game and returns to the main menu. During a game each action asks for confirmation. After the game has ended no confirmation is needed, so a rematch takes one click.
Trigger	The player clicks "Restart", "New Game" or "Quit".
Preconditions	PRE-16.1-1 A game is in progress or has just ended.

Item	Content
Postconditions	POST-16.1-1 After "Restart" the new game has the same settings and a clean state. POST-16.1-2 After "New Game" the mode selection screen is shown. POST-16.1-3 After "Quit" the main menu is shown.
Normal flow	1. The player clicks a button. 2. The system asks "Are you sure?" if the game is not finished. 3. The player confirms. 4. The Game Controller resets or clears the Game State. 5. The UI Layer shows the resulting screen.
Alternative flows	A1. The player cancels the confirmation. The game continues unchanged.
Exceptions	None.
Diagrams	Figure 5 (use case diagram).

5. Non-Functional Requirements

The non-functional requirements describe how well the Ludo game must work, not what it does. They match the architecture in Section 3: the whole game runs in the browser, so the limits below are about the browser and the player's computer. Each requirement has an ID, the reason for it and the member who wrote it.

5.1 Performance requirements

The game must feel instant, because players click quickly and the computer players must not slow the game down (REQ-PLAYER13, REQ-PLAYER14).

ID	Requirement	Rationale	Written by
NFR 5.1.1	After every player action (roll, move, restart, new game) the UI Layer shall redraw the board, pieces, dice and progress panel within 200 ms. Moving a piece shall be animated for at most 200 ms per square.	The board must stay easy to follow and respond immediately (REQ-PLAYER14).	Ahad Ali Saeed
NFR 5.1.2	A computer player shall roll and make its move within 1 second of its turn starting, and the whole turn of a computer player (including extra rolls) shall not take longer than 3 seconds.	The game must not slow down when computer players play (REQ-PLAYER13).	Sufyan Ahmad

5.2 Space requirements

ID	Requirement	Rationale	Written by
NFR 5.2.1	The game shall use at most 200 MB of browser memory. All files of the game (code, images, sounds) shall be at most 10 MB in total. The game shall not store any data on a server and shall keep the Game State only in the browser memory while the page is open.	The game must run on ordinary student laptops and load fast, and no account or server is needed.	Abdullah Ayub

5.3 Reliability requirements

ID	Requirement	Rationale	Written by
NFR 5.3.1	The game shall not crash or freeze during normal play. A full game of 4 players, including restart and new game, shall be playable without reloading the page. Wrong clicks (for example clicking a piece of another player or clicking while a computer plays) shall be ignored or answered with a message, and shall never stop the game.	The client expects a game that does not crash during normal play (D2, Section 3).	Sufyan Ahmad

5.4 Usability requirements

ID	Requirement	Rationale	Written by
NFR 5.4.1	A first-time player shall be able to start a game and make the first move without reading any instructions. The name of the player whose turn it is shall always be visible, and the pieces that can move shall always be highlighted. Colours shall never be the only way to show information: names and text labels shall also be used.	Casual players such as Hassan never read instructions and want to start in a few clicks (D2, Section 2).	Ahad Ali Saeed

5.5 Portability requirements

ID	Requirement	Rationale	Written by
NFR 5.5.1	The game shall run in the latest two versions of Google Chrome, Microsoft Edge and Mozilla Firefox on Windows 10 or later, Linux and macOS, without installation or administrator rights. The game shall be usable on screens at least 1024 pixels wide.	Other teams and the client must be able to open and test the game from a link on any computer.	Ahad Ali Saeed

5.6 Maintainability and extensibility requirements

The client expects that future features and rule changes are possible (D2, Section 3).

ID	Requirement	Rationale	Written by
NFR 5.6.1	The game rules shall be implemented only in the Rules Engine and shall not depend on the UI Layer. Values that can change, such as the track length, the safe squares, the number of pieces per player and the number of consecutive sixes that cancel, shall be stored as named settings in one place.	A rule change must need a change in one component only, and the rules must be testable without the screen.	Abdullah Ayub
NFR 5.6.2	The source code shall be stored in a shared GitHub repository with meaningful commit messages. Every component of Section 3 shall be a separate module with a short comment describing its input and output.	All members and the Scrum Coach must be able to follow the history and understand each component (D1).	Abdullah Ayub

5.7 Fairness requirements

ID	Requirement	Rationale	Written by
NFR 5.7.1	The Dice Module shall give every value from 1 to 6 with the same chance, using a software random number generator, and every roll shall be independent of earlier rolls. Computer players shall use the same dice and the same rules as human players, and shall not know the next dice value in advance.	Experienced players such as Mariam expect fair random dice and no cheating by the computer (D2, Section 2, REQ-PLAYER6).	Sufyan Ahmad

6. Traceability Matrix

Every D2 user story is covered by the 12 functional and 9 non-functional requirements of this document.

D2 user story	Covering D3 requirement	Responsible author
REQ-PLAYER1 (Start Game)	REQ-PLAYER1.1 (Start a new game)	Ahad Ali Saeed
REQ-PLAYER2, 3, 4 (Mode, Seats, Names)	REQ-PLAYER2.1 (Choose game mode and seats)	Ahad Ali Saeed
REQ-PLAYER5 (Read Rules)	REQ-PLAYER5.1 (View the rules summary)	Ahad Ali Saeed
REQ-PLAYER6 (Roll Dice)	REQ-PLAYER6.1 (Roll the dice)	Sufyan Ahmad

D2 user story	Covering D3 requirement	Responsible author
REQ-PLAYER7 (Move Piece)	REQ-PLAYER7.1 (Move a piece)	Sufyan Ahmad
REQ-PLAYER8 (Turn Indicator)	REQ-PLAYER8.1 (Show and manage turns)	Abdullah Ayub
REQ-PLAYER9 (Invalid Move Feedback)	REQ-PLAYER9.1 (Reject invalid moves)	Sufyan Ahmad
REQ-PLAYER10 (Capture Piece)	REQ-PLAYER10.1 (Capture an opponent piece)	Abdullah Ayub
REQ-PLAYER11 (Six Activation & Re-roll)	REQ-PLAYER6.1, REQ-PLAYER7.1	Sufyan Ahmad
REQ-PLAYER12 (Progress)	REQ-PLAYER12.1 (Show progress and update the screen)	Abdullah Ayub
REQ-PLAYER13 (CPU Speed)	REQ-PLAYER13.1 (Computer player turn), NFR 5.1.2	Sufyan Ahmad
REQ-PLAYER14 (UI Speed)	REQ-PLAYER12.1, NFR 5.1.1	Abdullah Ayub / Ahad Ali Saeed
REQ-PLAYER15 (Announce Winner)	REQ-PLAYER15.1 (Announce the winner)	Ahad Ali Saeed
REQ-PLAYER16, 17, 18 (Restart, New Game, Quit)	REQ-PLAYER16.1 (Restart, start a new game and quit)	Abdullah Ayub

Appendix A: Changelog

Version	Date	Change
v1	2026-11-04	Initial version. Based on D2: Ludo rules, game modes and seat setup, 12 functional requirements covering REQ-PLAYER1 to REQ-PLAYER18, and 9 non-functional requirements. Includes the explicit consecutive-sixes rule, component and data-flow diagrams, traceability matrix and group contact information.

Appendix B: Individual Contributions

Course rule: at least 2 functional and 3 non-functional requirements per member. Each of us wrote 4 functional and 3 non-functional requirements.

Name	Individual contributions
Ahad Ali Saeed	Functional: REQ-PLAYER1.1, REQ-PLAYER2.1, REQ-PLAYER5.1, REQ-PLAYER15.1 Non-functional: NFR 5.1.1, NFR 5.4.1, NFR 5.5.1
Sufyan Ahmad	Functional: REQ-PLAYER6.1, REQ-PLAYER7.1, REQ-PLAYER9.1, REQ-PLAYER13.1 Non-functional: NFR 5.1.2, NFR 5.3.1, NFR 5.7.1
Abdullah Ayub	Functional: REQ-PLAYER8.1, REQ-PLAYER10.1, REQ-PLAYER12.1, REQ-PLAYER16.1 Non-functional: NFR 5.2.1, NFR 5.6.1, NFR 5.6.2

[1] Standard Ludo rules, as summarised in D2 and at https://en.wikipedia.org/wiki/Ludo_(board_game).